

CAP Application Architecture Guide

Backend Structure Patterns for Interns

This document describes three approaches to structuring the backend of a SAP Cloud Application Programming Model (CAP) project. Each section explains the folder layout, the role of each layer, and how data flows from an incoming request down to the database. The goal is to give interns a clear mental model before writing their first line of code.

1. Background — How CAP Works

CAP is SAP's framework for building enterprise-grade services. Every CAP project has two kinds of files that work together:

- CDS files (.cds) — define your data model, service endpoints, and annotations. This is the contract.
- JavaScript files (.js) — implement the behavior behind that contract: event handlers, business logic, database access.

CAP handles a lot automatically — OData protocol, pagination, authentication hooks, database abstraction. Your job as a developer is to write the business logic that sits behind the service events (READ, CREATE, UPDATE, DELETE, and custom actions).

The core question this document answers

CAP does not enforce how you organize your .js handler code. That freedom is also a responsibility. The following sections describe three ways teams have answered that question — each with a different philosophy.

2. Approach 1 — MVC-Inspired Flat Layer Architecture

2.1 Philosophy

This approach draws from the Model-View-Controller pattern common in frameworks like Express, Spring, and NestJS. The idea is to create a clear horizontal stack of layers: a router layer, a controller layer, a service layer, a repository layer, and utilities. Each layer has one defined responsibility and can only communicate downward.

2.2 Folder Structure

```
srv/  
├── service.cds           ← All CDS service and entity definitions  
├── service.js           ← Router only: registers CAP event handlers  
├── controllers/  
│   ├── table.controller.js ← Orchestrates a request, calls services/repos  
│   ├── declaration.controller.js  
│   └── user.controller.js  
├── services/  
│   ├── table.service.js   ← Business logic for the table domain  
│   ├── calculation.service.js  
│   └── validation.service.js  
├── repos/  
│   ├── table.repo.js      ← All database access for tables  
│   └── declaration.repo.js  
└── utils/  
    ├── formatter.js       ← Pure stateless helper functions  
    └── date.util.js
```

2.3 Layer Responsibilities

service.js — The Router

This file has one job: register CAP event handlers and immediately delegate to the matching controller. It should contain no logic whatsoever. Think of it as a telephone switchboard — it only routes calls.

```
const tableController = require('./controllers/table.controller');  
const declController  = require('./controllers/declaration.controller');  
  
module.exports = cds.service.impl(async function () {  
  this.on('getTableRows', tableController.getTableRows);  
  this.before('CREATE', 'Tables', tableController.validateTable);  
  this.on('READ', 'Declarations', declController.readDeclarations);  
});
```

controllers/ — The Orchestrators

A controller receives the CAP request object and decides which services or repos to call and in what order. It does not contain business rules or SQL. It is purely coordination logic: call this, then call that, then return the result.

```
const tableService = require('../services/table.service');
const tableRepo    = require('../repos/table.repo');

const getTableRows = async (req) => {
  const { tableCode } = req.data;
  const table = await tableRepo.findByCode(tableCode);
  return tableService.buildRowsWithGroups(table);
};

const validateTable = async (req) => {
  await tableService.validate(req.data);
};

module.exports = { getTableRows, validateTable };
```

services/ — Business Logic

Services contain the rules and calculations that make your application meaningful. They do not talk to the database directly — they call repos for that. A service can call other services or utils, but it should never import a controller.

```
const tableRepo    = require('../repos/table.repo');
const formatter     = require('../utils/formatter');

const buildRowsWithGroups = (table) => {
  // business logic: group rows, apply rules, compute totals
  return table.rows.map(row => formatter.formatRow(row));
};

const validate = async (data) => {
  if (!data.code) throw new Error('Table code is required');
};

module.exports = { buildRowsWithGroups, validate };
```

repos/ — Database Access

Every database query lives in a repo file. Repos use CDS SELECT, INSERT, UPDATE, and DELETE. They return plain data — no business logic, no formatting, just raw results from the database. This makes it easy to swap out or mock the database in tests.

```
const { Tables, Cells } = cds.entities('liasse.fiscale');
```

```
const findByCode = async (code) => {
  return SELECT.one.from(Tables).where({ code });
};

const findCellsByTableId = async (tableId) => {
  return SELECT.from(Cells).where({ table_ID: tableId });
};

module.exports = { findByCode, findCellsByTableId };
```

utils/ — Stateless Helpers

Utils are pure functions with no dependency on CAP or the database. Things like date formatting, number rounding, string transformations, and currency conversion belong here. They can be imported by any layer.

2.4 Request Flow

How a request travels through Approach 1

CAP Event → service.js (route) → controller (orchestrate) → service (business logic) → repo (database) → response

3. Approach 2 — CDS-Driven Impl Layer Architecture

3.1 Philosophy

This approach follows CAP's idiomatic conventions. Instead of one central service.js that routes everything, each CDS service file references its own implementation file in an impl/ folder. CAP loads these impl files automatically. Each impl file delegates to a domain service, which calls a repo. Everything related to one business domain lives together.

3.2 Folder Structure

```
srv/
├── table.service.cds      ← CDS service definition for tables
├── declaration.service.cds ← CDS service definition for declarations
├── impl/
│   ├── table.impl.js     ← CAP handler file for table service
│   └── declaration.impl.js ← CAP handler file for declaration service
├── domain/
│   ├── table.domain.service.js ← Business logic for the table domain
│   └── declaration.domain.service.js
```

```
└─ repo/
   └─ table.repo.js      ← Database access for table entities
   └─ declaration.repo.js
```

3.3 Layer Responsibilities

CDS files — Service Definitions

Each CDS file defines one service. The `@impl` annotation tells CAP where to find the corresponding JavaScript implementation file. CAP will auto-load it — no manual registration needed.

```
@impl: 'srv/impl/table.impl'
service TableService {
  action getTableRows(tableCode: String) returns array of TableRow;
  entity Tables as projection on lf.Tables;
  entity Cells as projection on lf.Cells;
}
```

impl/ — The Thin Handler Layer

The impl file is the entry point for CAP events. Its only job is to register event handlers and immediately delegate to the domain service. It should never contain business logic, SQL, or conditional rules. Think of it as the receptionist — it receives the call and passes it to the right person.

```
const tableDomainService = require('../domain/table.domain.service');

module.exports = cds.service.impl(async function () {
  this.on('getTableRows', async (req) => {
    return tableDomainService.getRowsWithGroups(req.data.tableCode);
  });

  this.before('CREATE', 'Tables', async (req) => {
    await tableDomainService.validate(req.data);
  });
});
```

domain/ — Business Logic

The domain service is where all meaningful business decisions are made. It receives clean inputs from the impl layer, applies rules, performs calculations, and calls the repo for data. It knows about business concepts like 'a table has groups' or 'a cell has rules' — but it does not know about HTTP, OData, or SQL.

```
const tableRepo = require('../repo/table.repo');

const getRowsWithGroups = async (tableCode) => {
  const table = await tableRepo.findByCode(tableCode);
  const groups = await tableRepo.findGroupsByTableId(table.ID);
  const cells = await tableRepo.findCellsByTableId(table.ID);
  // build the hierarchical structure...
  return buildHierarchy(groups, cells);
};

const validate = async (data) => {
  if (!data.code) throw new Error('Table code is required');
};

module.exports = { getRowsWithGroups, validate };
```

repo/ — Database Access

Same principle as Approach 1: one repo per domain, containing all SELECT / INSERT / UPDATE / DELETE statements for that domain's entities. The domain service calls the repo; the repo calls the database; nothing else.

```
const { Tables, Cells, RowGroups } = cds.entities('liasse.fiscale');

const findByCode = async (code) => {
  return SELECT.one.from(Tables).where({ code });
};

const findGroupsByTableId = async (tableId) => {
  return SELECT.from(RowGroups).where({ table_ID: tableId }).orderBy('sortOrder');
};

const findCellsByTableId = async (tableId) => {
  return SELECT.from(Cells).where({ table_ID: tableId }).orderBy('rowNumber');
};

module.exports = { findByCode, findGroupsByTableId, findCellsByTableId };
```

3.4 Request Flow

How a request travels through Approach 2

CAP Event → impl/ (delegate) → domain.service (business logic) → repo (database) → response

4. Approach 3 — Feature-Grouped Domain Architecture (Proposed)

4.1 Philosophy

Both previous approaches organize files horizontally — all controllers together, all services together, all repos together. This third approach organizes files vertically, by feature. Every file that belongs to the 'table' domain lives in one folder: its CDS file, its impl file, its domain service, and its repo. A shared/ folder handles anything used by more than one domain.

This is inspired by the Screaming Architecture principle: when you open the project folder, it should immediately tell you what the application does ('tables', 'declarations', 'users') rather than how it is built ('controllers', 'services', 'repos').

4.2 Folder Structure

```
srv/
├── table/
│   ├── table.service.cds           ← CDS definition for table domain
│   ├── table.impl.js              ← CAP event handlers
│   ├── table.domain.service.js    ← Business logic
│   └── table.repo.js              ← Database access
├── declaration/
│   ├── declaration.service.cds
│   ├── declaration.impl.js
│   ├── declaration.domain.service.js
│   └── declaration.repo.js
├── user/
│   ├── user.service.cds
│   ├── user.impl.js
│   ├── user.domain.service.js
│   └── user.repo.js
└── shared/
    ├── services/
    │   └── calculation.service.js ← Logic used by multiple domains
    ├── external/
    │   └── tax-api.client.js      ← External API integrations
    └── utils/
        ├── formatter.js
        └── date.util.js
```

4.3 Layer Responsibilities

Domain folders — Self-Contained Feature Modules

Each domain folder contains everything needed to implement that feature end-to-end. A developer working on the table feature opens the table/ folder and finds all four files they need. They rarely need to look elsewhere.

The internal structure of each domain folder mirrors Approach 2: a thin impl file delegates to a domain service, which calls a repo. The difference is purely organizational — files are co-located by domain instead of scattered across global layer folders.

```
// table/table.impl.js
const tableDomainService = require('./table.domain.service');

module.exports = cds.service.impl(async function () {
  this.on('getTableRows', async (req) => {
    return tableDomainService.getRowsWithGroups(req.data.tableCode);
  });
});
```

```
// table/table.domain.service.js
const tableRepo = require('./table.repo');
const calculationService = require('../shared/services/calculation.service');

const getRowsWithGroups = async (tableCode) => {
  const table = await tableRepo.findByCode(tableCode);
  const groups = await tableRepo.findGroupsByTableId(table.ID);
  const cells = await tableRepo.findCellsByTableId(table.ID);
  return calculationService.buildHierarchy(groups, cells);
};

module.exports = { getRowsWithGroups };
```

```
// table/table.repo.js
const { Tables, Cells, RowGroups } = cds.entities('liasse.fiscale');

const findByCode = async (code) => {
  return SELECT.one.from(Tables).where({ code });
};

const findGroupsByTableId = async (tableId) => {
  return SELECT.from(RowGroups).where({ table_ID: tableId }).orderBy('sortOrder');
};

const findCellsByTableId = async (tableId) => {
  return SELECT.from(Cells).where({ table_ID: tableId }).orderBy('rowNumber');
};

module.exports = { findByCode, findGroupsByTableId, findCellsByTableId };
```

shared/ — Cross-Domain Logic

The shared folder has three sub-folders with distinct purposes:

- `shared/services/` — business logic that is legitimately used by more than one domain. For example, a calculation engine used by both the table domain and the declaration domain.
- `shared/external/` — all integrations with external APIs or third-party services. Each external dependency gets its own client file with a clean interface. Domain services call these clients; they never call the external API directly.
- `shared/utils/` — stateless pure helper functions that have no dependency on CAP, the database, or any domain concept.

Rule of thumb for `shared/`

A file only moves to `shared/` when it is genuinely needed by two or more domains. Do not move something to `shared/` as speculation — this leads to over-engineering. Start with it inside the domain and promote it only when the second consumer appears.

4.4 Request Flow

How a request travels through Approach 3

CAP Event → `domain/impl` (delegate) → `domain.service` (business logic, may call `shared/`) → `domain.repo` (database) → response

5. Rules That Apply to All Three Approaches

Regardless of which approach your team adopts, the following rules should always hold. Violating them is the most common source of technical debt in CAP projects.

5.1 Layer Rules

- The `impl` / router layer never contains business logic. It receives a request and delegates immediately.
- The `service` / domain layer never writes SQL or calls CDS entities directly. It calls repos for that.
- The `repo` layer never contains business rules, conditional logic, or formatted output. It runs a query and returns raw data.
- Utils are pure functions. They take inputs, return outputs, and have no side effects.
- No layer imports from a layer above it. Controllers do not import from `impl`. Repos do not import from `services`.

5.2 Dependency Rules

- External API calls always go through a dedicated client in `external/` or `utils/`. A domain service never calls `fetch()` or `axios` directly.
- A domain service never imports directly from another domain's service. Shared logic goes to a shared location.
- CDS entities are always accessed through `repos`, never directly in a service, controller, or `impl` file.

5.3 File Size Rules

- If an `impl` file exceeds 40 lines, business logic has leaked in. Move it to the domain service.
- If a service file exceeds 200 lines, it is doing too much. Split it by responsibility.
- If a `repo` file contains an `if` statement that is not purely for query construction, business logic has leaked in. Move it to the service.

6. Which Approach to Use

The choice depends on project size and team familiarity, not on any single approach being objectively superior.

Approach 1 is a good fit when the team is new to CAP but experienced with MVC frameworks like Express or Spring. It uses familiar concepts and makes onboarding fast. The tradeoff is that it does not follow CAP's own conventions, which can cause confusion when reading SAP documentation or community resources.

Approach 2 is a good fit when the team is learning CAP from scratch and wants to follow SAP's idiomatic patterns. The auto-loading of `impl` files, the naming conventions, and the per-service structure align with how SAP writes its own examples. It works well for medium-sized projects with a stable set of domains.

Approach 3 is a good fit for larger projects or teams that expect the domain count to grow over time. By co-locating all files for a domain, it becomes straightforward to add a new domain, remove one, or hand off one domain to a different developer. The `shared/` folder makes cross-cutting concerns explicit rather than implicit.

Starting recommendation for new interns

If you are starting a new project today without a pre-existing team convention, Approach 3 is the safest long-term choice. It combines the idiomatic CAP patterns of Approach 2 with a feature-first organization that scales naturally as the application grows.